Plasma Science & Fusion Center

Machine Learning Working Group

Data Science Division

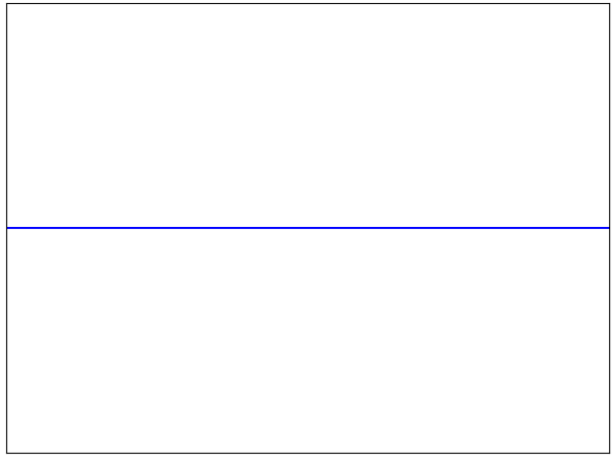
Tutorial 3 – Gaussian Processes

Aaron Ho

02/26/2026

Gaussian Processes can be treated like probabilistic curve fitting

Training points (known data)	x_i, y_i			
Hyperparameters (fitting coefficients)	ϕ	$\rightarrow$	Prediction outputs	y
Prediction inputs	x			



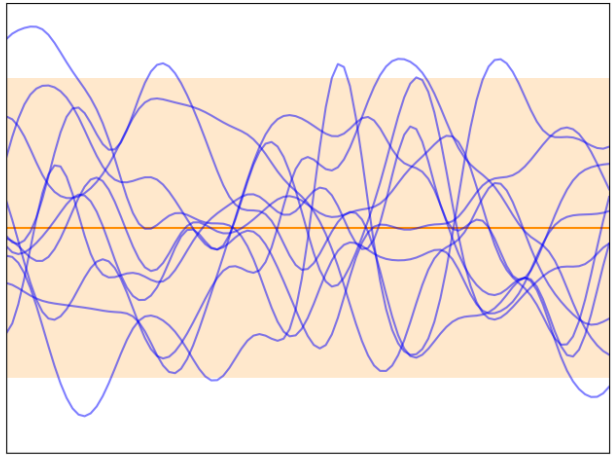
Gaussian Processes can be treated like probabilistic curve fitting

Training points (known data) X_i, Y_i ish

Hyperparameters (fitting coefficients) Θ sorta

Prediction inputs X kinda

Prediction outputs Y maybe

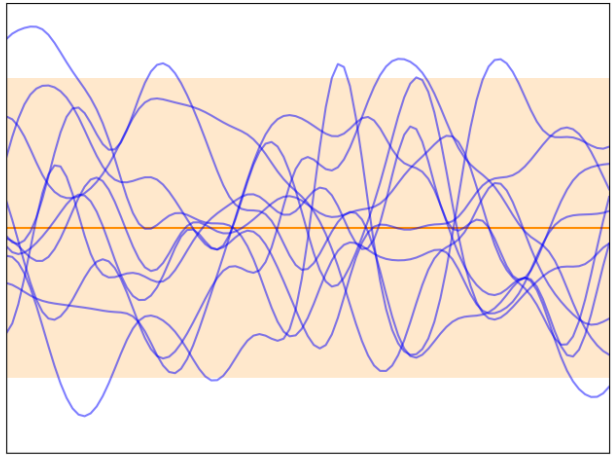


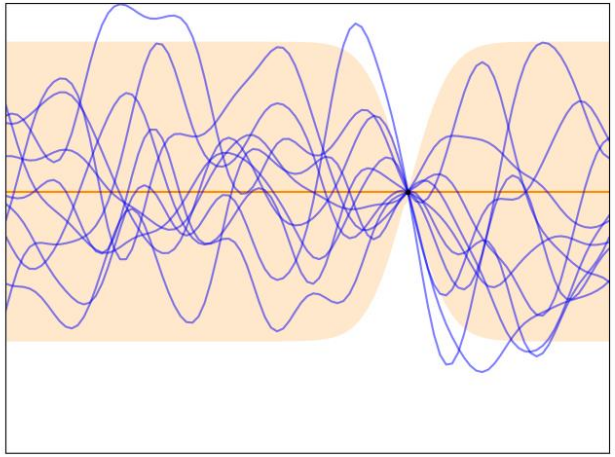
Gaussian Processes can be treated like probabilistic curve fitting

Training points (known data) X_i, Y_i

Hyperparameters (fitting coefficients) Θ → Prediction outputs Y

Prediction inputs X



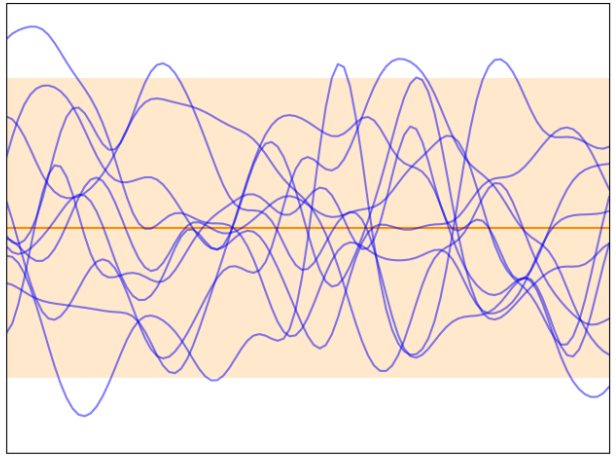


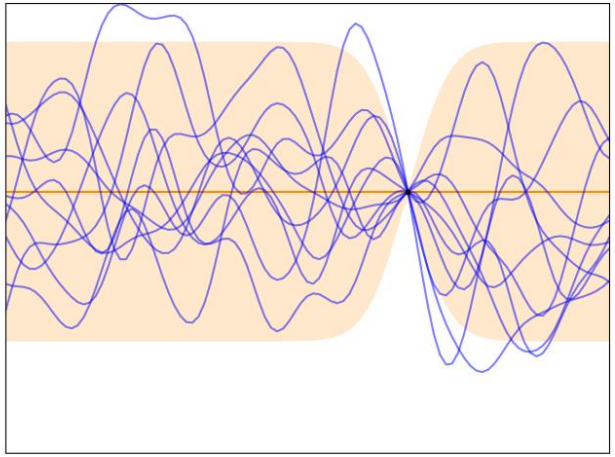
Gaussian Processes can be treated like probabilistic curve fitting

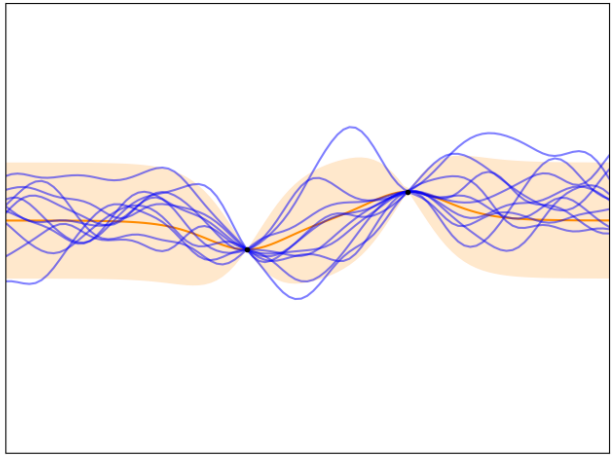
Training points (known data) X_i, Y_i

Hyperparameters (fitting coefficients) Θ → Prediction outputs Y

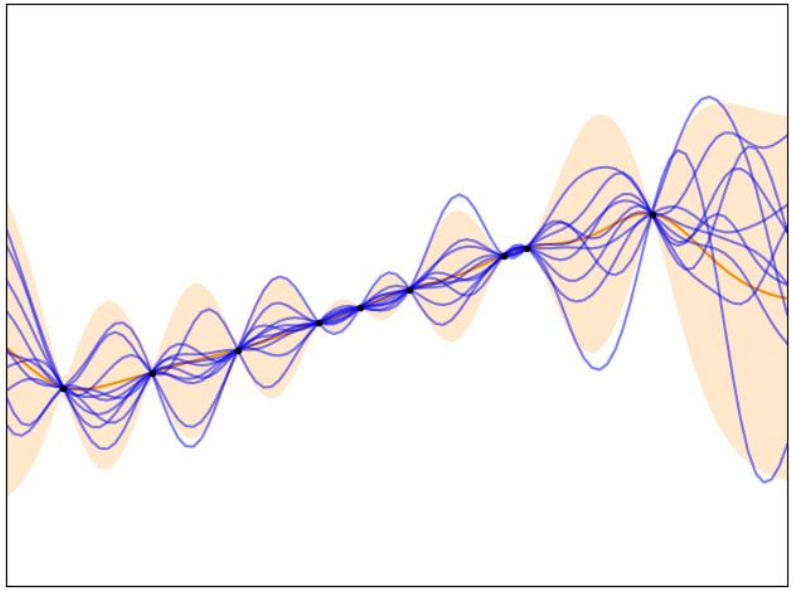
Prediction inputs X



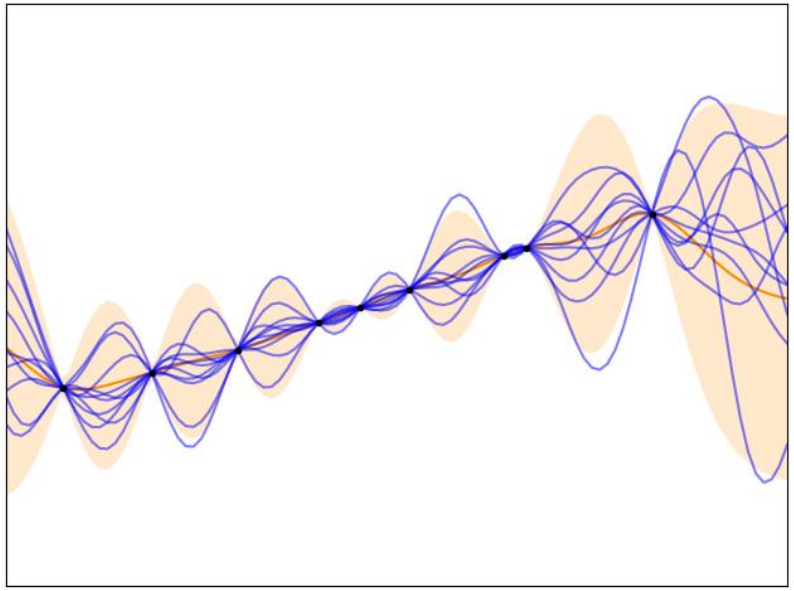


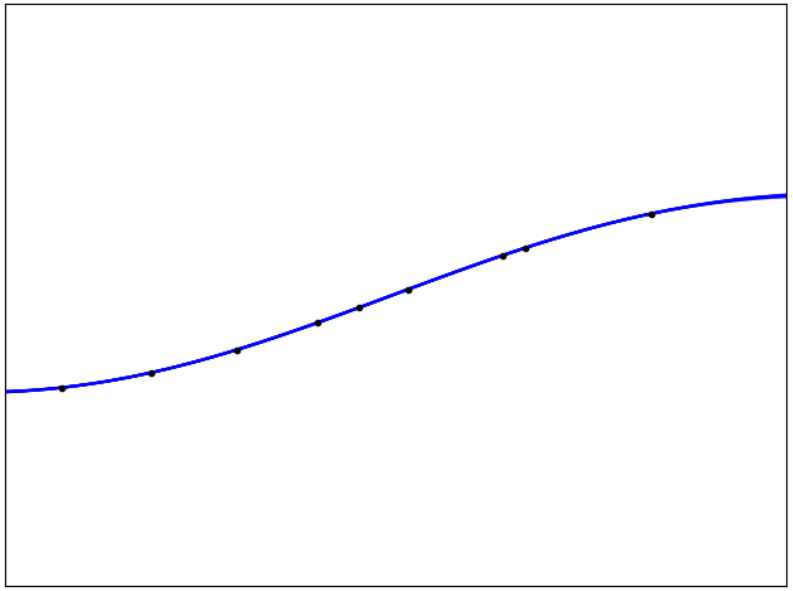


Fitting is done by maximizing log-marginal-likelihood (LML)

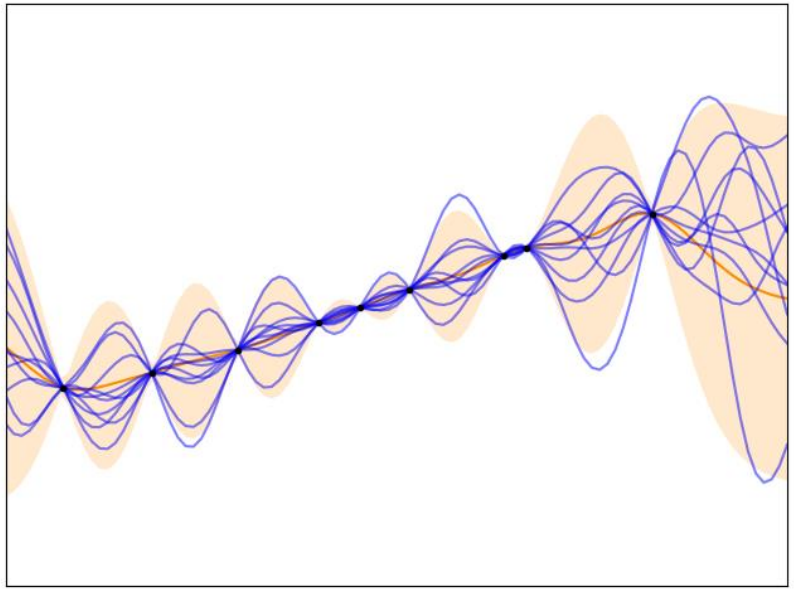


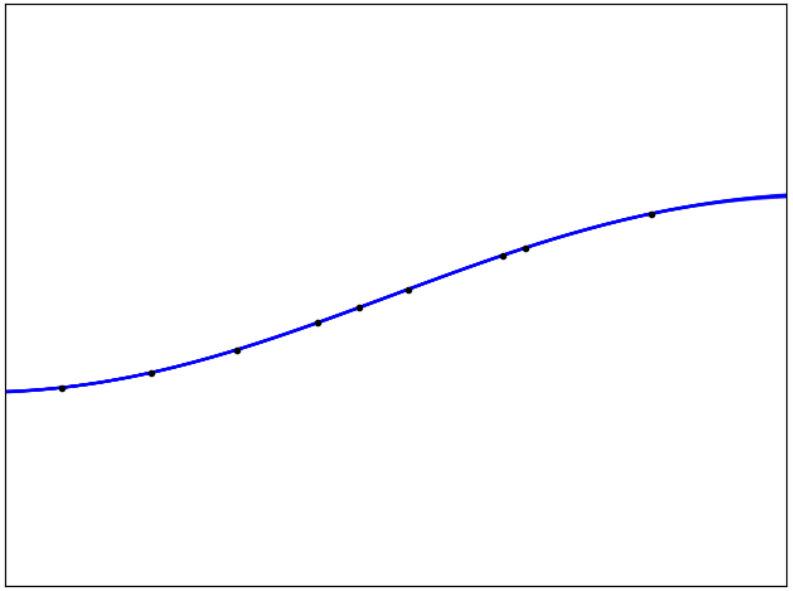
Fitting is done by maximizing log-marginal-likelihood (LML)





Fitting is done by maximizing log-marginal-likelihood (LML)

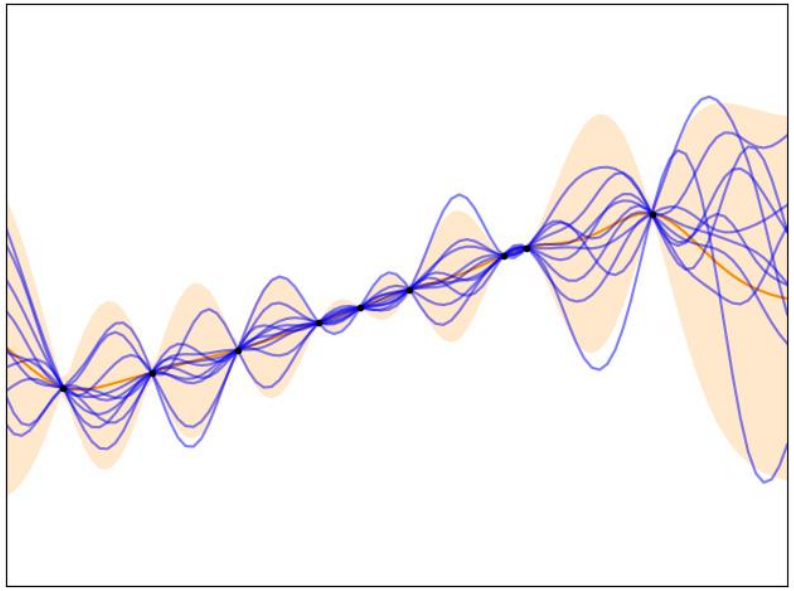


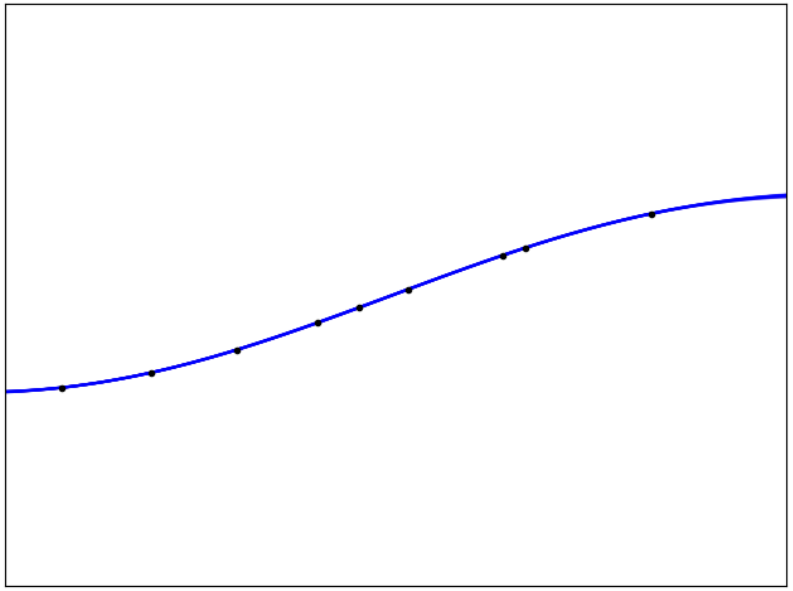


LML comes from Bayes' Theorem: total probability of $\boldsymbol{X_i}, \boldsymbol{Y_i}$ integrated over $\boldsymbol{\Theta}$-distribution

$$\mathrm{LML} = \underbrace{-\boldsymbol{Y_i}^T \boldsymbol{K}^{-1} \boldsymbol{Y_i}}_{\text{goodness-of-fit}} \underbrace{-\lambda \log|\boldsymbol{K}|}_{\text{complexity}} \underbrace{-n \log 2\pi}_{\text{normalization}}$$

Fitting is done by maximizing log-marginal-likelihood (LML)





LML comes from Bayes' Theorem: total probability of $\boldsymbol{X_i}, \boldsymbol{Y_i}$ integrated over $\boldsymbol{\Theta}$-distribution

$$\mathrm{LML} = \underbrace{-\boldsymbol{Y_i}^T \boldsymbol{K}^{-1} \boldsymbol{Y_i}}_{\text{goodness-of-fit}} \underbrace{-\lambda \log|\boldsymbol{K}|}_{\text{complexity}} \underbrace{-n \log 2\pi}_{\text{normalization}}$$

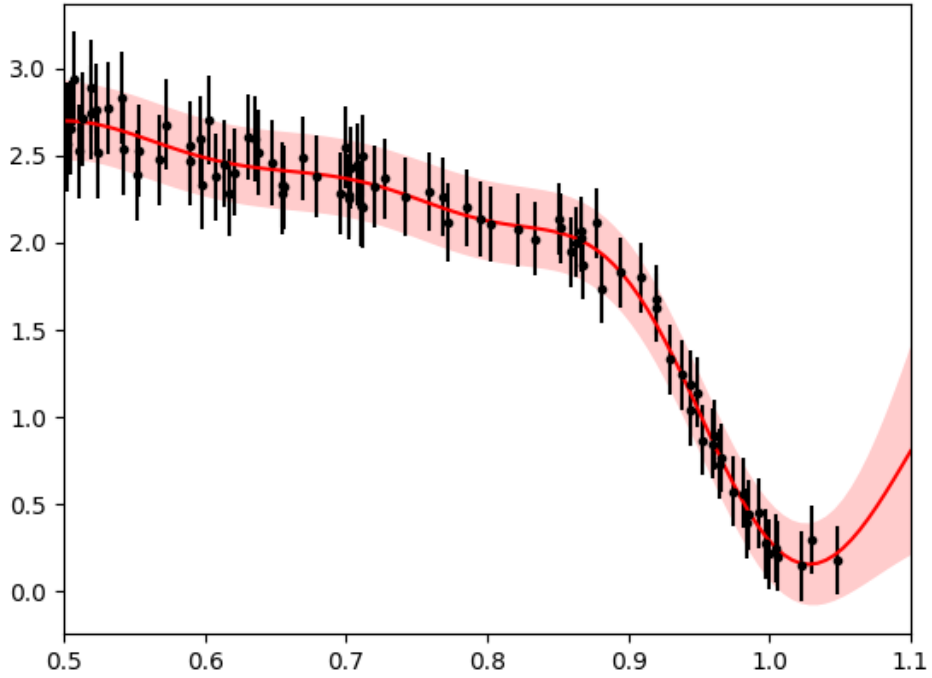
Since $\boldsymbol{\Theta}$ are probabilistic, Monte Carlo sampling should be used to get true output distribution

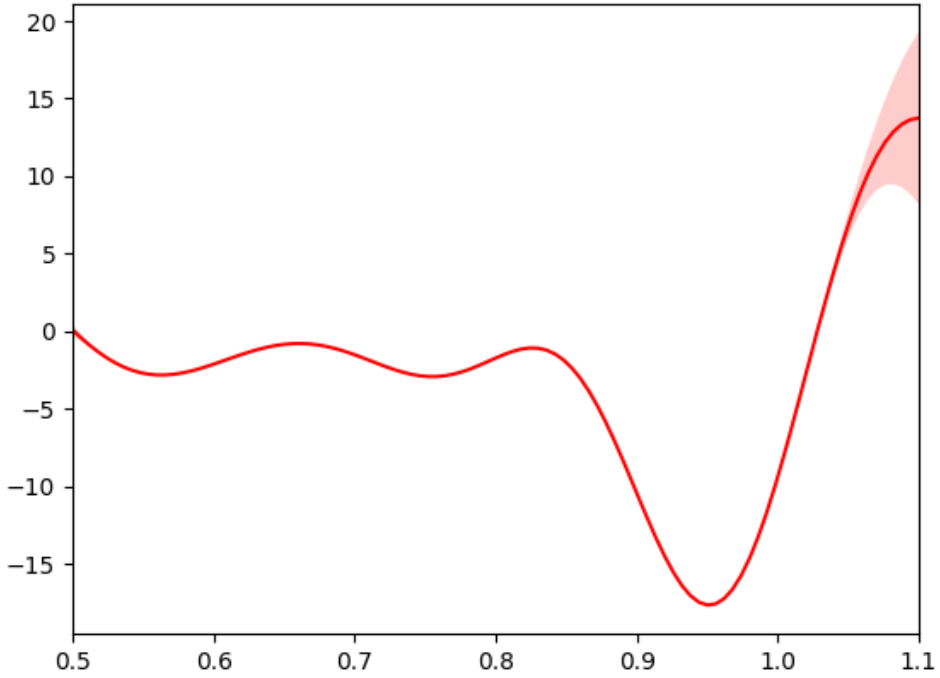
- Though all distributions are Gaussian, so evaluating only at most probable $\boldsymbol{\Theta}$ is often good enough
- Maximum *a posteriori* (MAP) solution

Examples

`gpytorch_curvefit.py`

- Loads sample 1-input 1-output dataset with error bars, fits GP (from gpytorch) on that data
- Plots data and fit distribution (left) and fit derivative distribution (right)

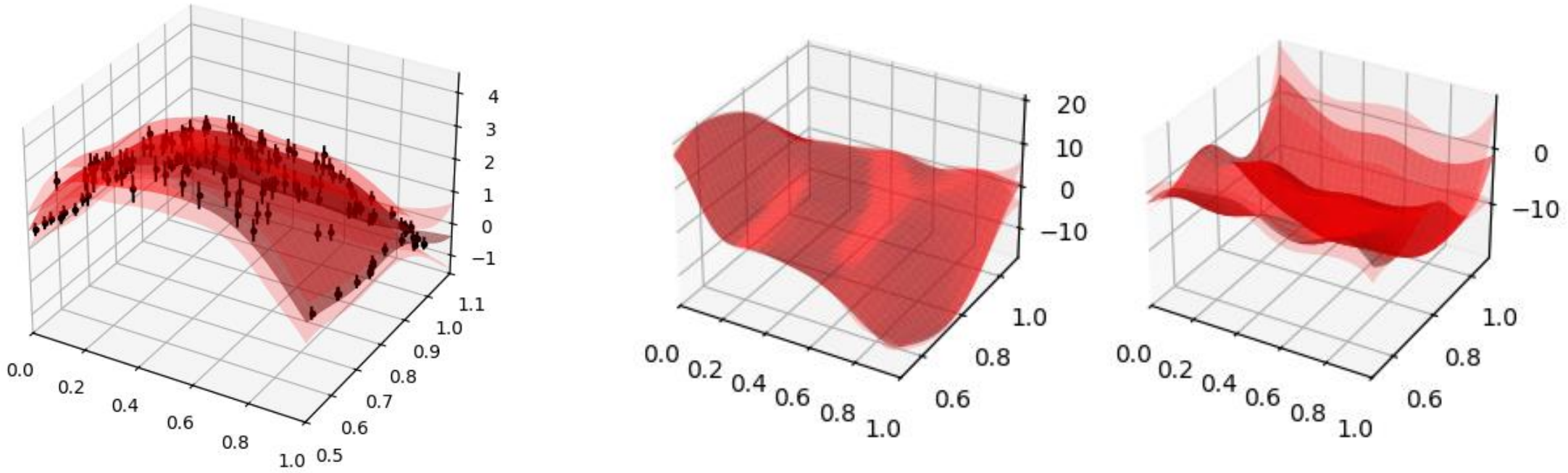




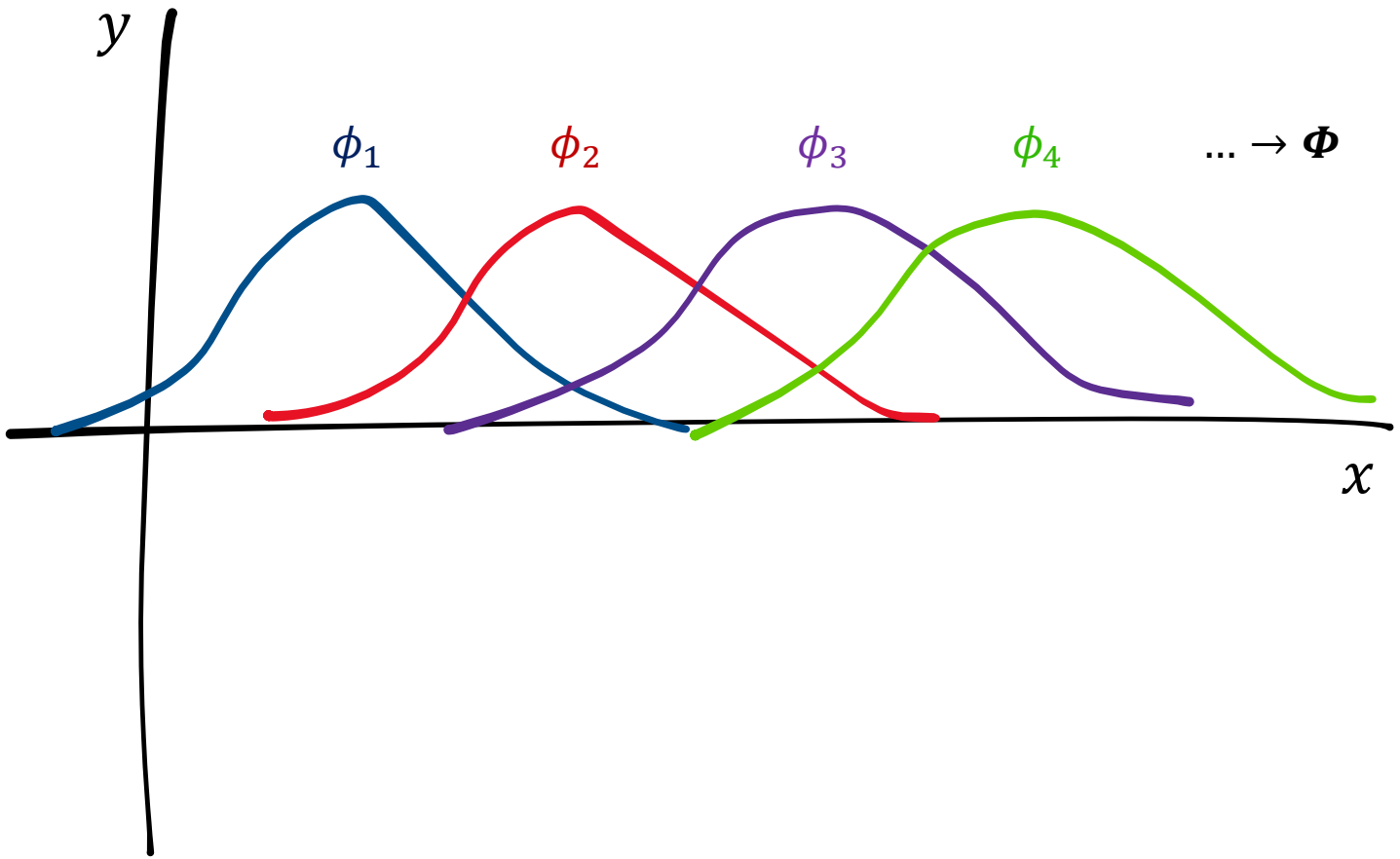
Examples

`gpytorch_surfacefit.py`

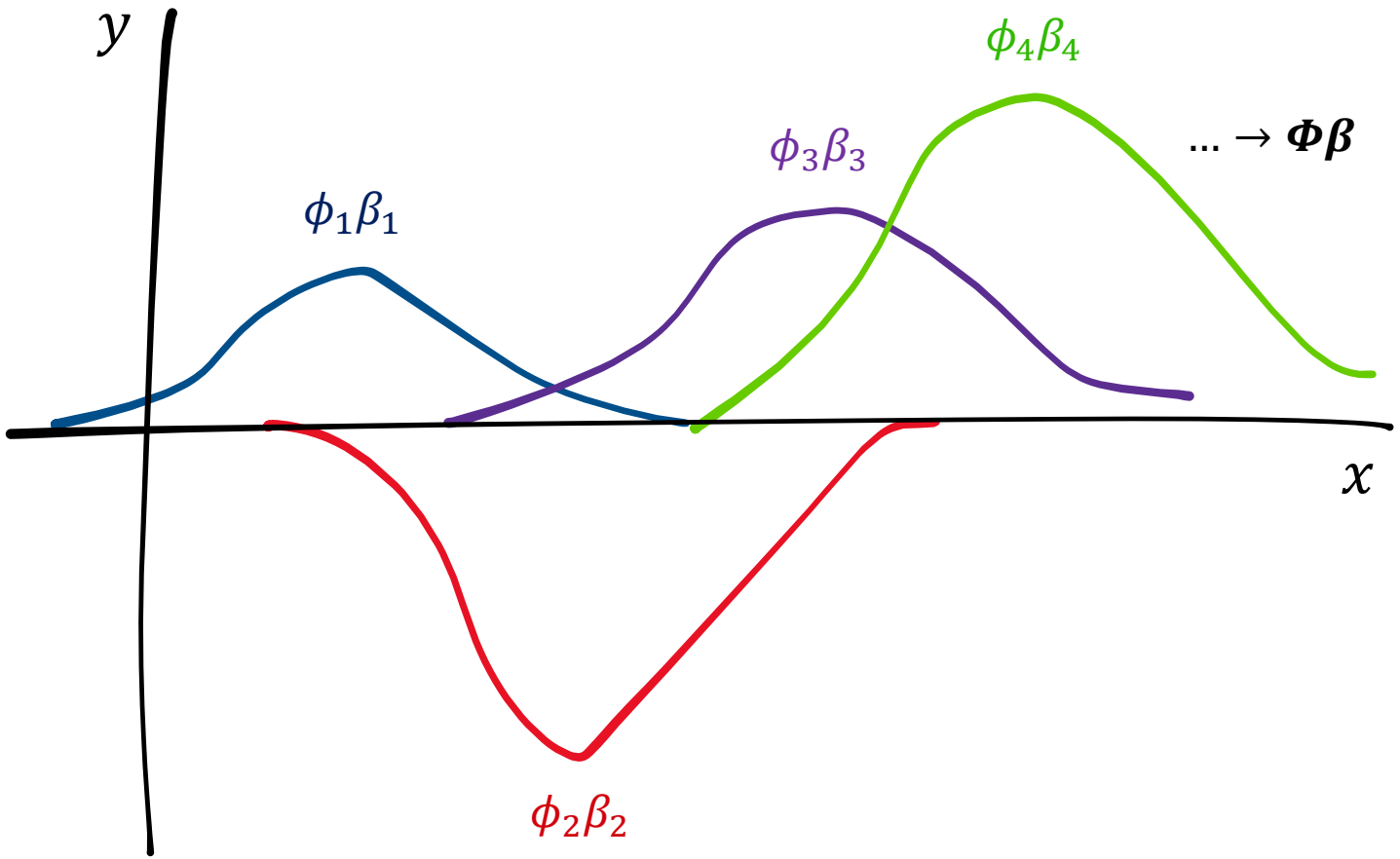
- Loads sample 2-input 1-output dataset with error bars, fits GP (from gpytorch) on that data
- Plots data and fit distribution (left) and fit derivative distribution (right)
- 2 plots for derivatives to reflect partial derivatives w.r.t. each input



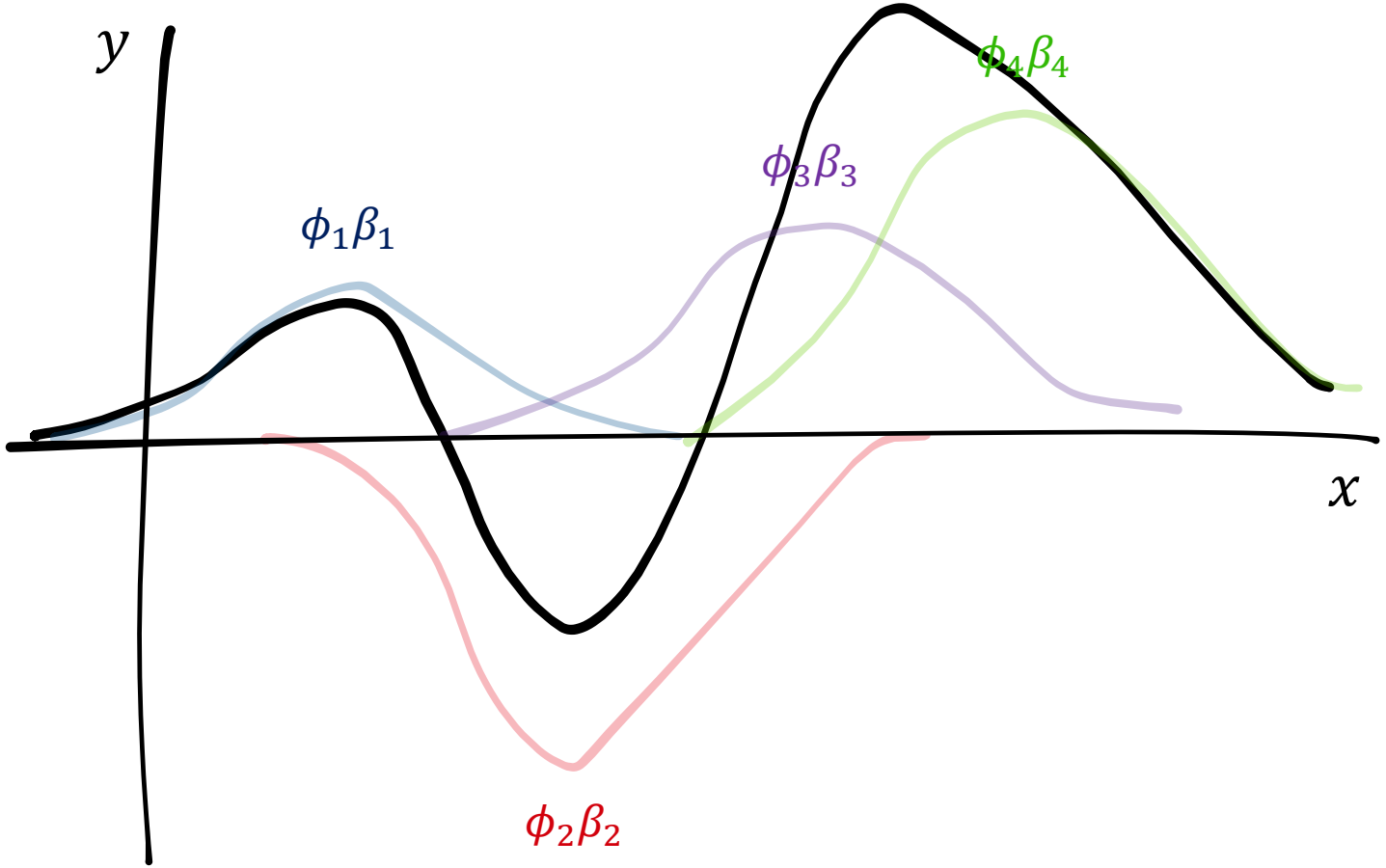
Using basis functions for curve fitting



Using basis functions for curve fitting

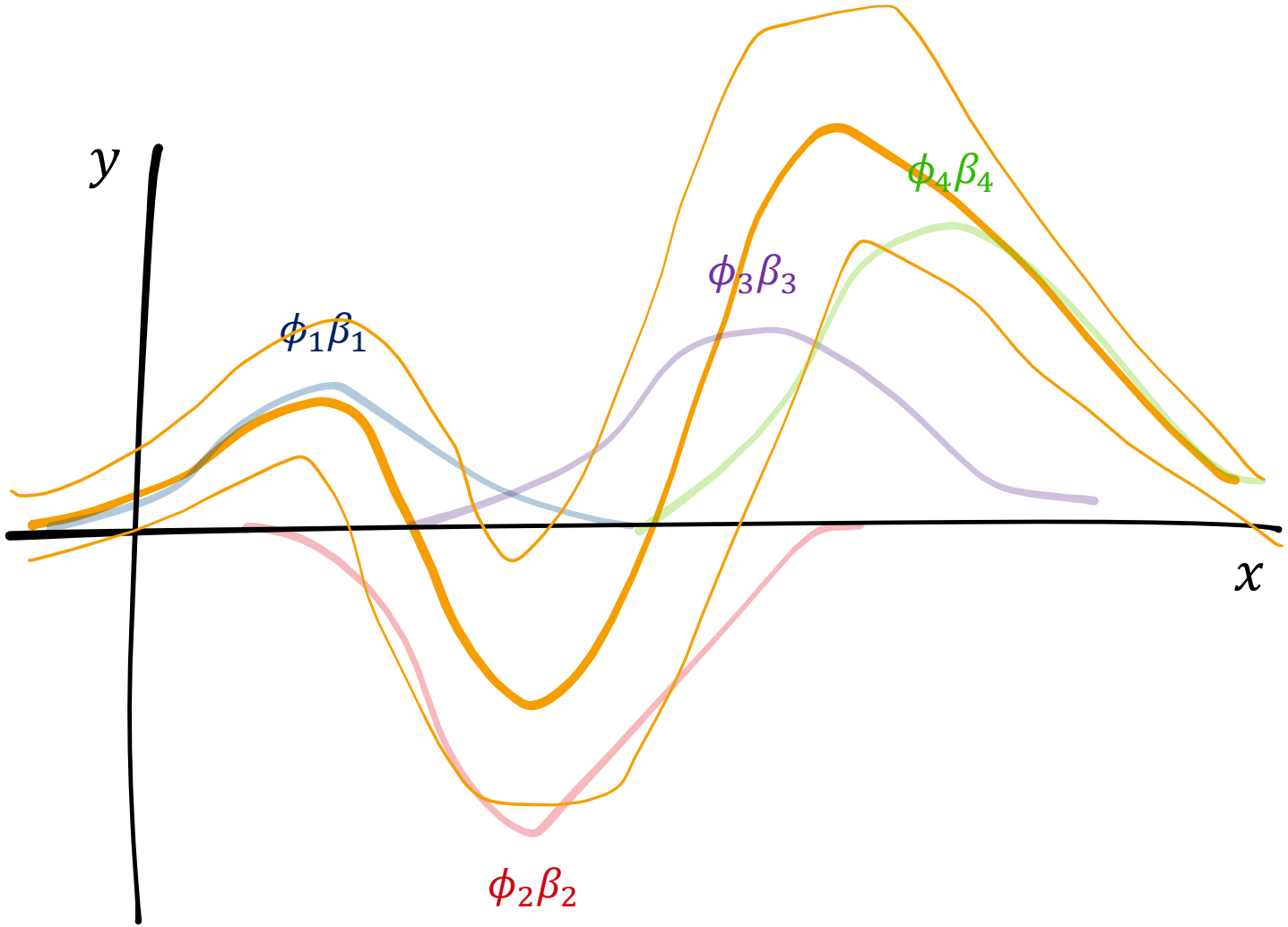


Using basis functions for curve fitting



$$\boldsymbol{Y} = \boldsymbol{\Phi}(\boldsymbol{X})\,\boldsymbol{\beta}$$

Gaussian Process use same principles, except via Bayes' Theorem



$$\boldsymbol{Y} = \boldsymbol{\Phi}(\boldsymbol{X})\,\boldsymbol{\beta}$$

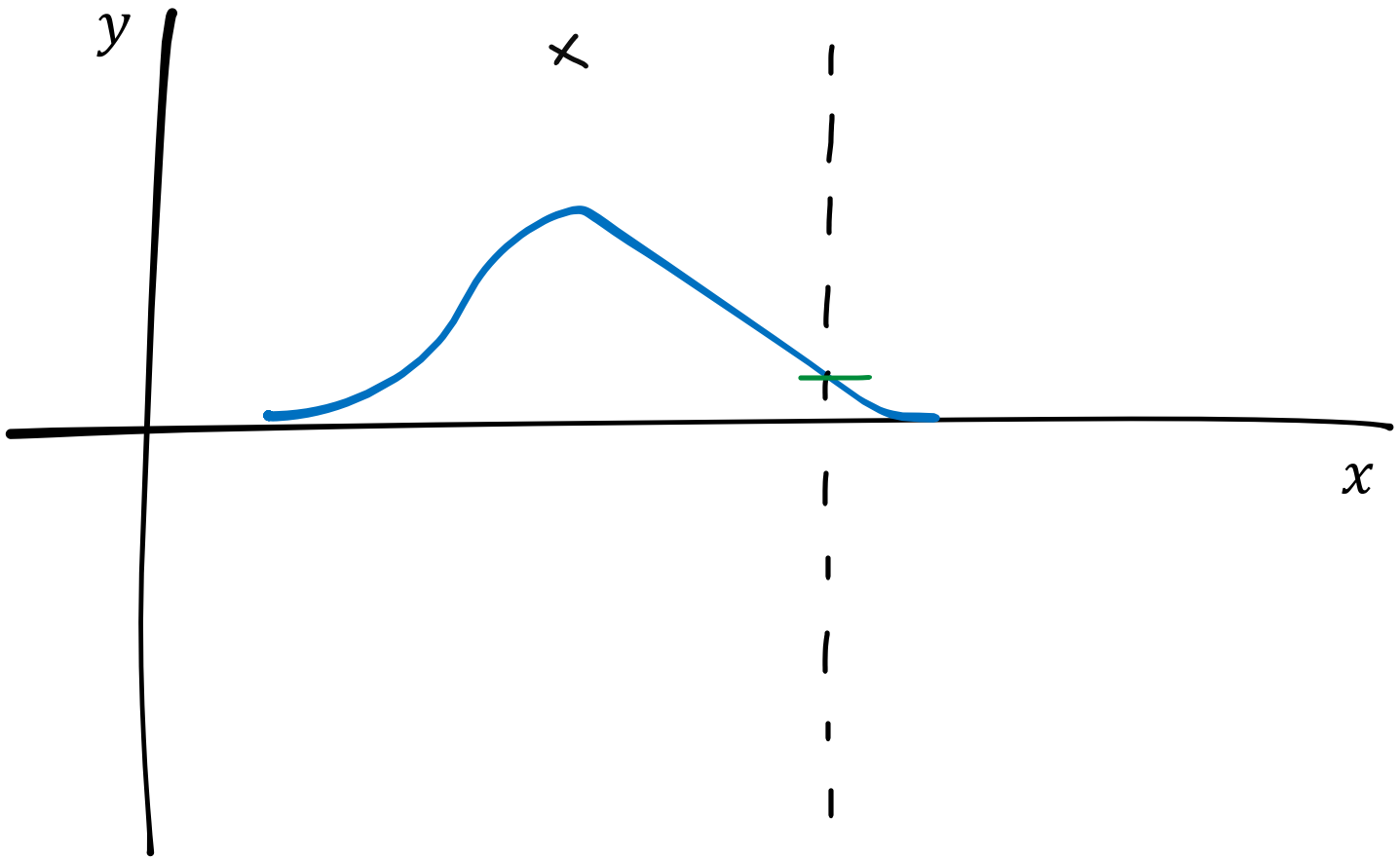
After shift to probabilities

$$\boldsymbol{Y} = \boldsymbol{K}(\boldsymbol{X},\boldsymbol{X_i})\,\boldsymbol{K}^{-1}(\boldsymbol{X_i},\boldsymbol{X_i})\,\boldsymbol{Y_i}$$

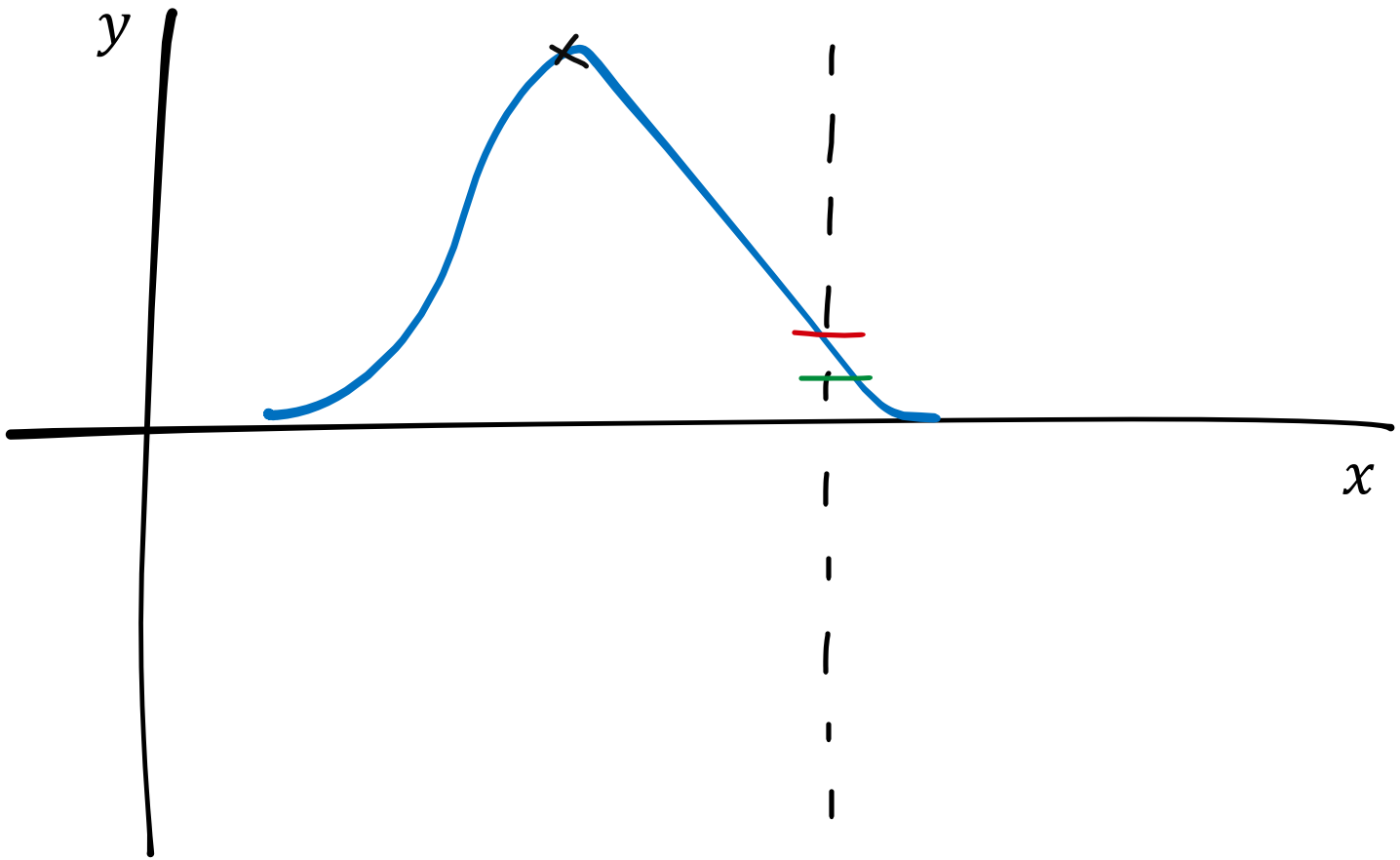
$$\boldsymbol{K}(\boldsymbol{X},\boldsymbol{X}) = \boldsymbol{\Phi}(\boldsymbol{X})\,\boldsymbol{\beta}\,\boldsymbol{\beta}^{\boldsymbol{T}}\,\boldsymbol{\Phi}^{\boldsymbol{T}}(\boldsymbol{X})$$

also known as the "kernel"

Gaussian Process extend curve fitting to probability distributions

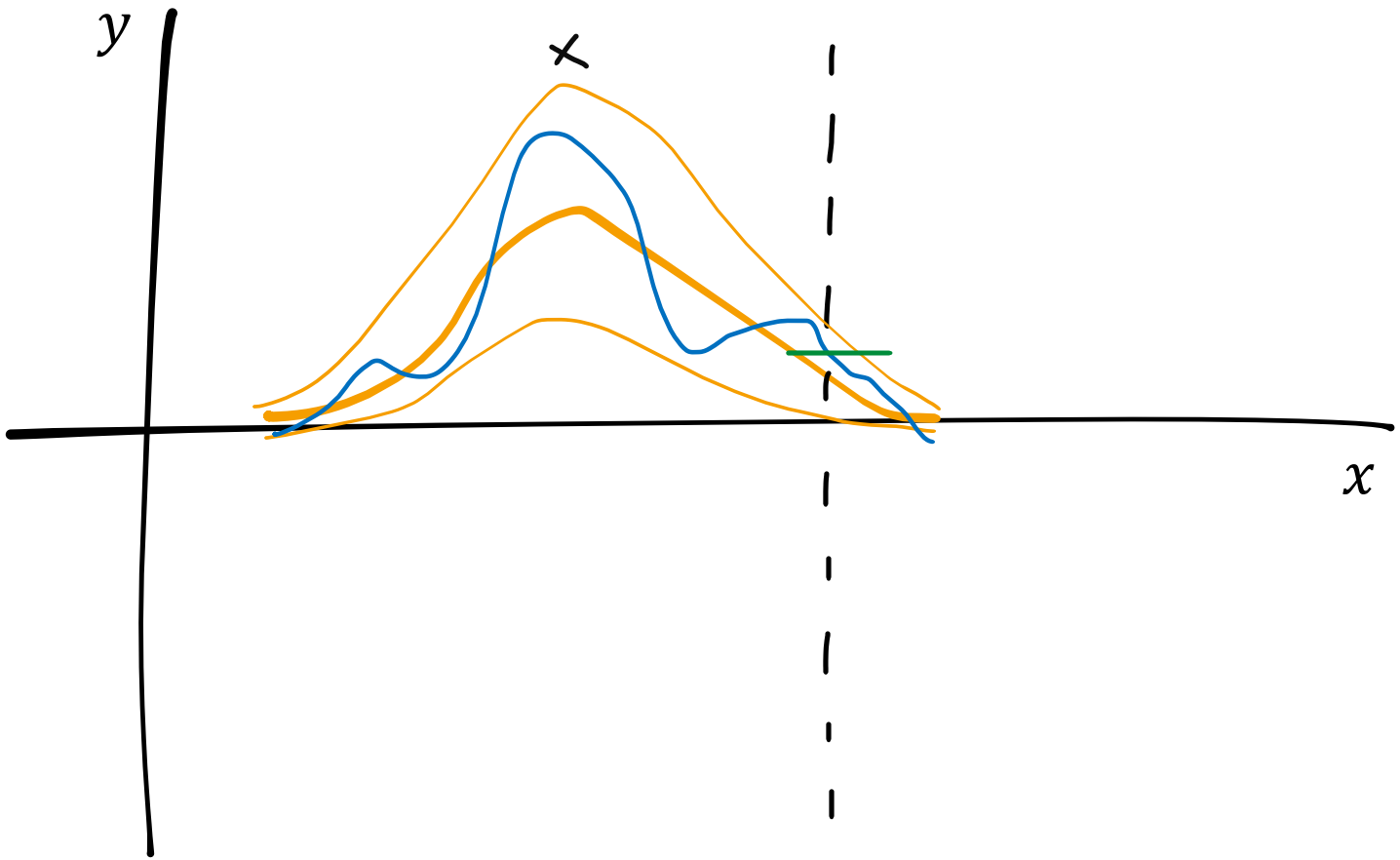


Gaussian Process extend curve fitting to probability distributions



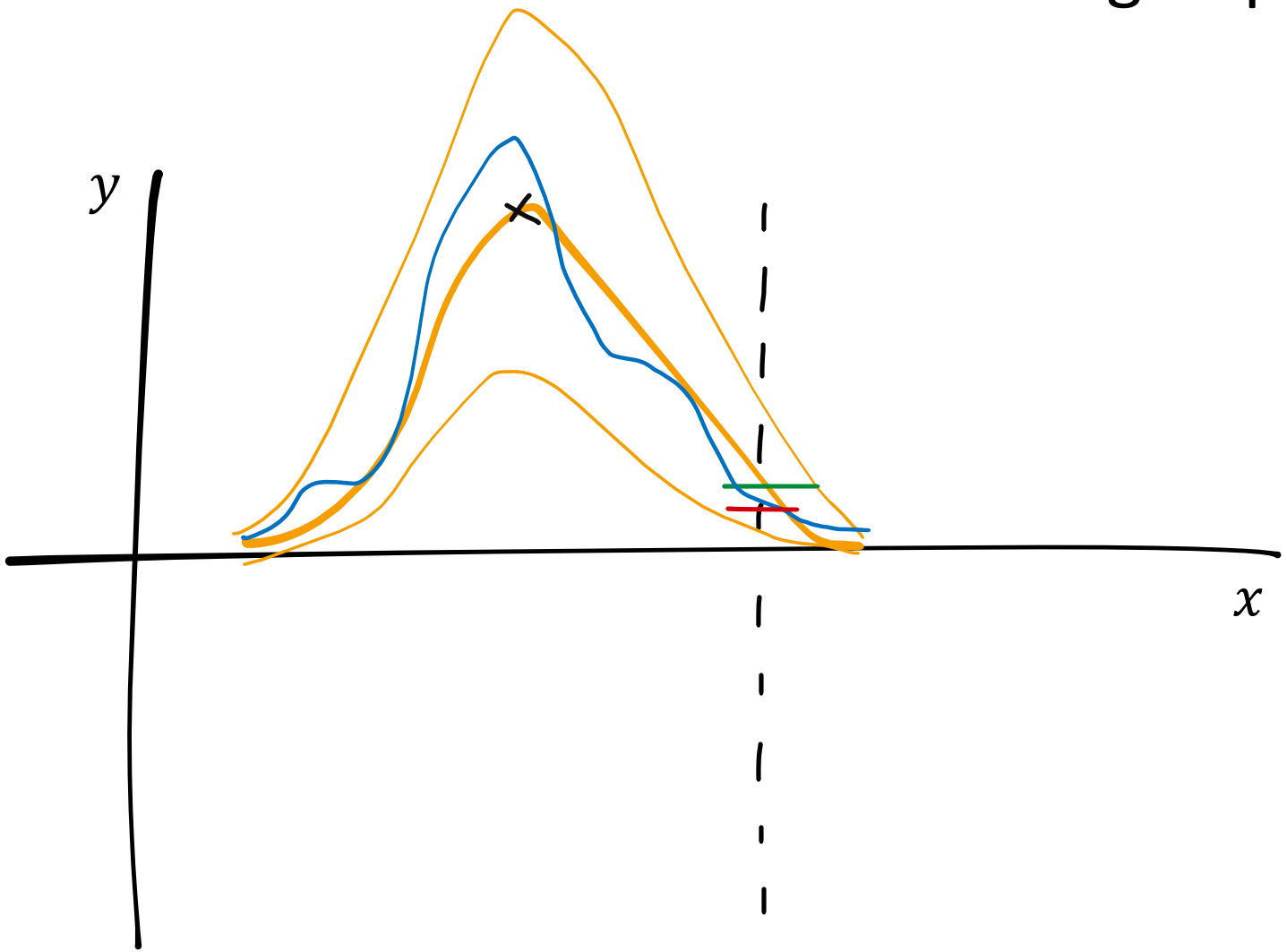
- Modifying β to match at one x_i, y_i impacts the prediction at every other x

Gaussian Process extend curve fitting to probability distributions



- Modifying β to match at one x_i, y_i impacts the prediction at every other x

Gaussian Process extend curve fitting to probability distributions



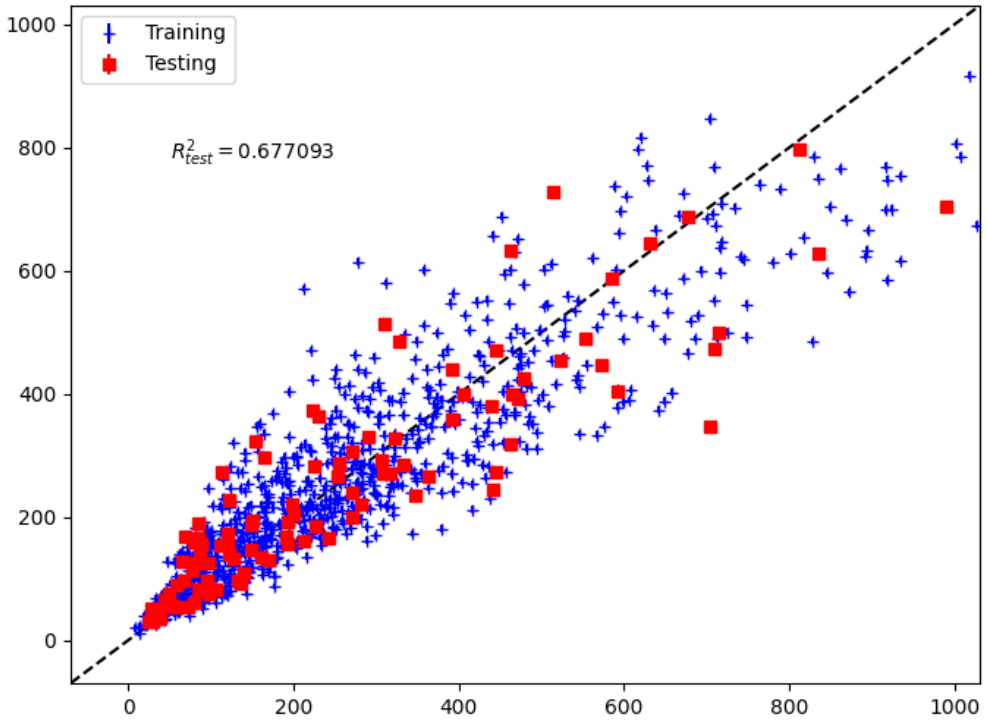
- Modifying β to match at one x_i, y_i impacts the prediction at every other x
- With probabilistic ϕ, this impact is not deterministic
  - Captured with covariance

    $\mathrm{Cov}(x, x) = E[XY] - E[X]E[Y]$
  - Effectively evaluated per pair of training points to give $\boldsymbol{K}$
- Gaussian $\boldsymbol{\Phi}$ gives symmetric $\boldsymbol{K}$ which decreases with distance

Practical session

`gpytorch_surrogatefit.py`

- Loads sample 10-input 1-output dataset with error bars, fits GP (from gpytorch) on that data
- Visualization is more difficult due to higher input dimensionality
- Can be used as a template for attempting a fit on your own data set



For those without a data set:

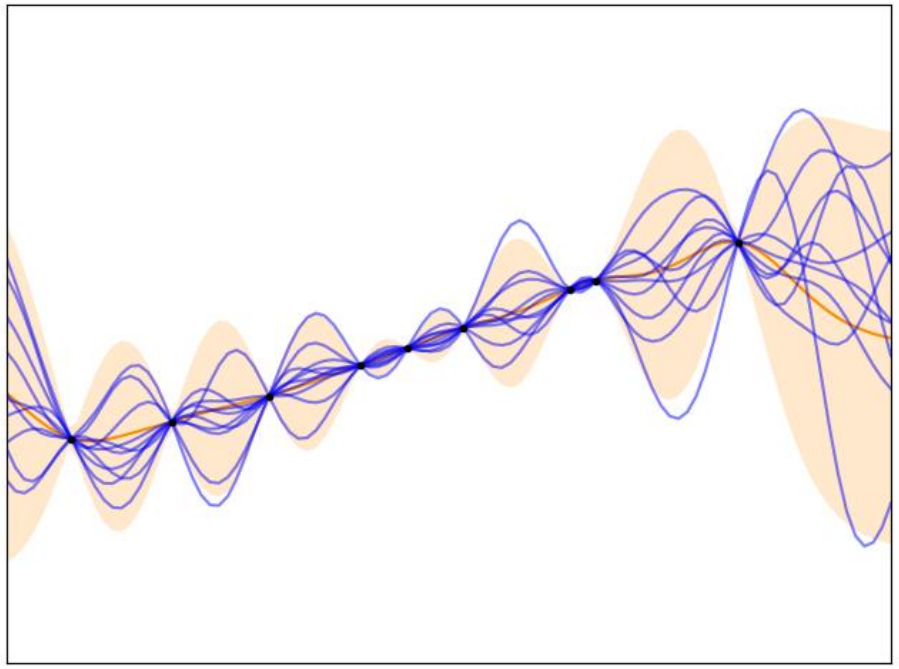
- Implement early stopping in 10-D fit script
- Improve the 10-D fit via data or gpytorch exploration
- Participate in Q&A

Advanced topics (limited material – mostly discussion)

1. GP sampling for Monte-Carlo-based uncertainty quantification
2. Kernel creation / selection
3. Outlier handling

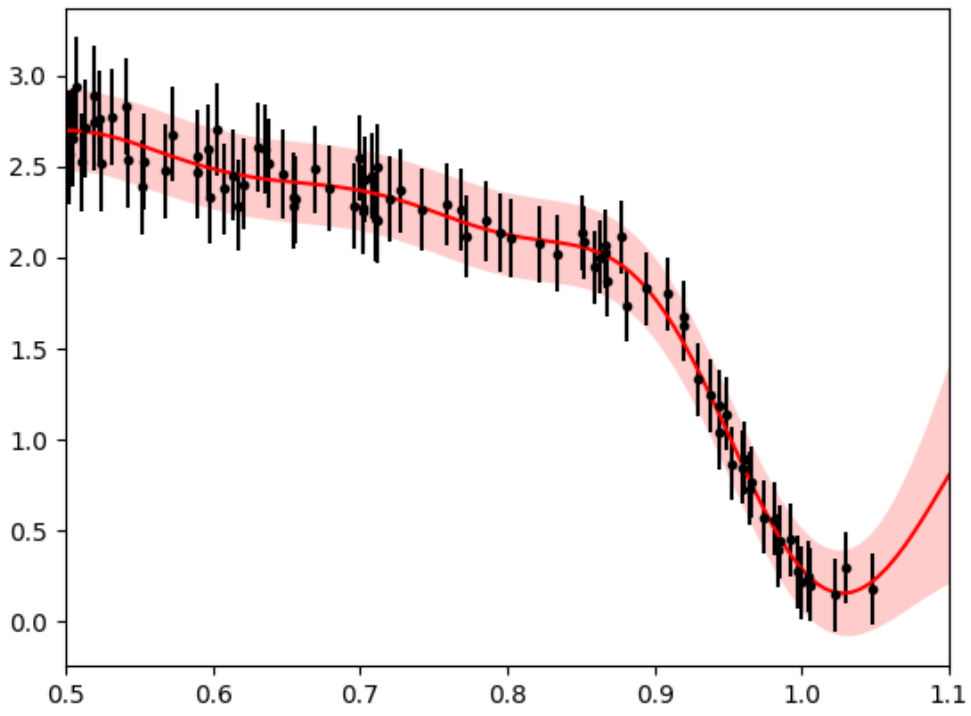
Sampling from GP for uncertainty quantification

- Deterministic curves are often needed to perform additional calculations, so sampling GP is useful



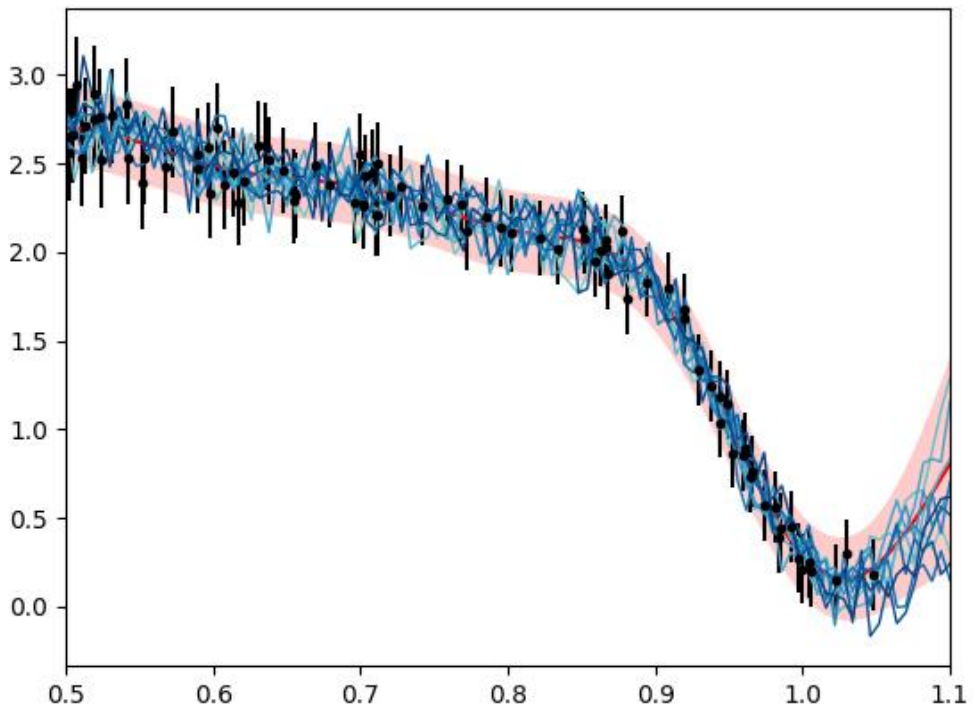
Sampling from GP for uncertainty quantification

- Deterministic curves are often needed to perform additional calculations, so sampling GP is useful



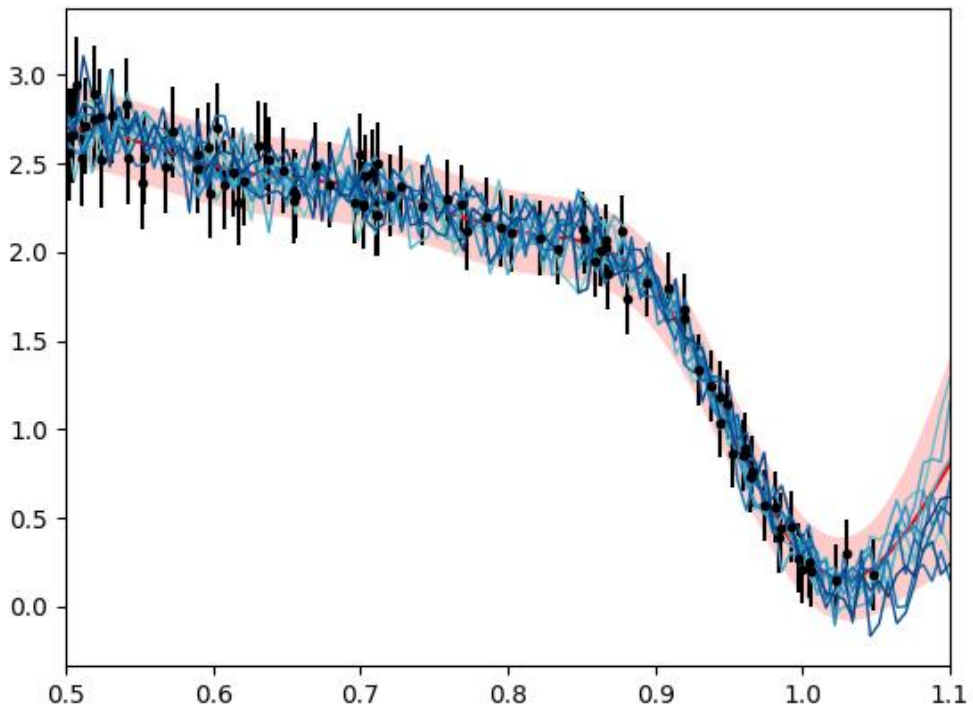
Sampling from GP for uncertainty quantification

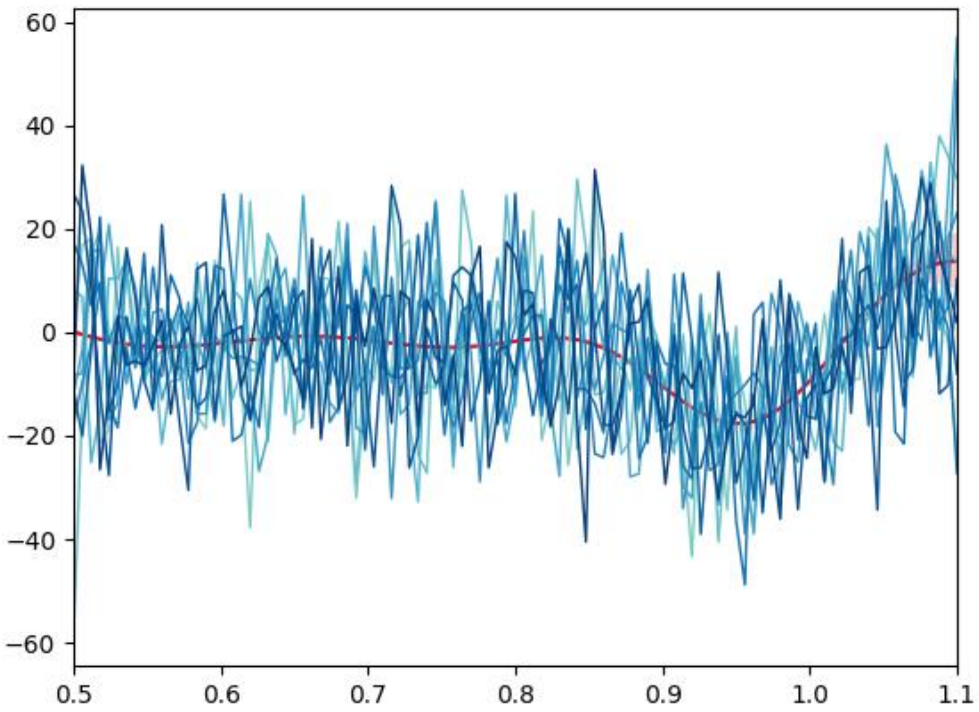
- Deterministic curves are often needed to perform additional calculations, so sampling GP is useful
- Sampling directly from some GP implementations give weird results (due to treatment of “noise”)



Sampling from GP for uncertainty quantification

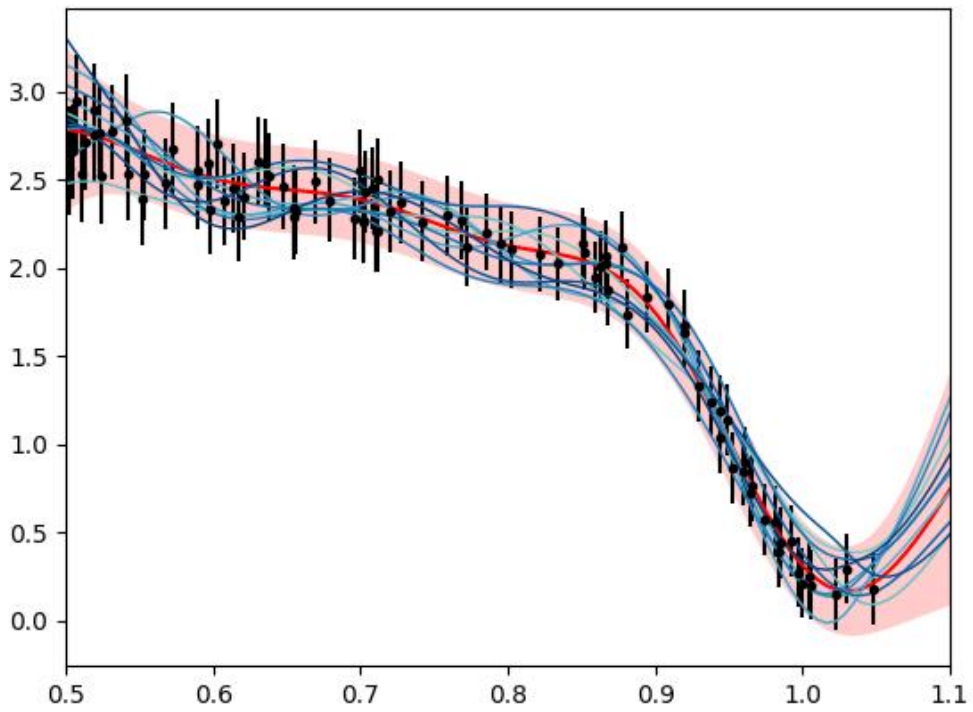
- Deterministic curves are often needed to perform additional calculations, so sampling GP is useful
- Sampling directly from some GP implementations give weird results (due to treatment of “noise”)

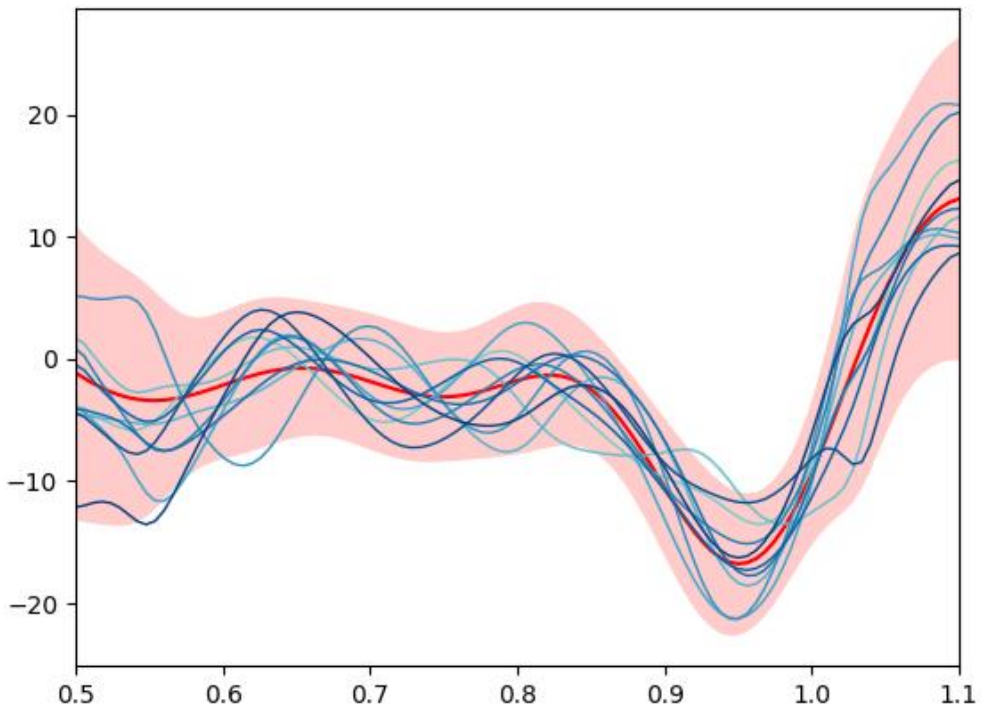




Sampling from GP for uncertainty quantification

- Deterministic curves are often needed to perform additional calculations, so sampling GP is useful
- Sampling directly from some GP implementations give weird results (due to treatment of “noise”)
- Running `mkgp_curvefit.py` (my personal GP package) treats uncertainties properly at cost of speed





Kernel selection

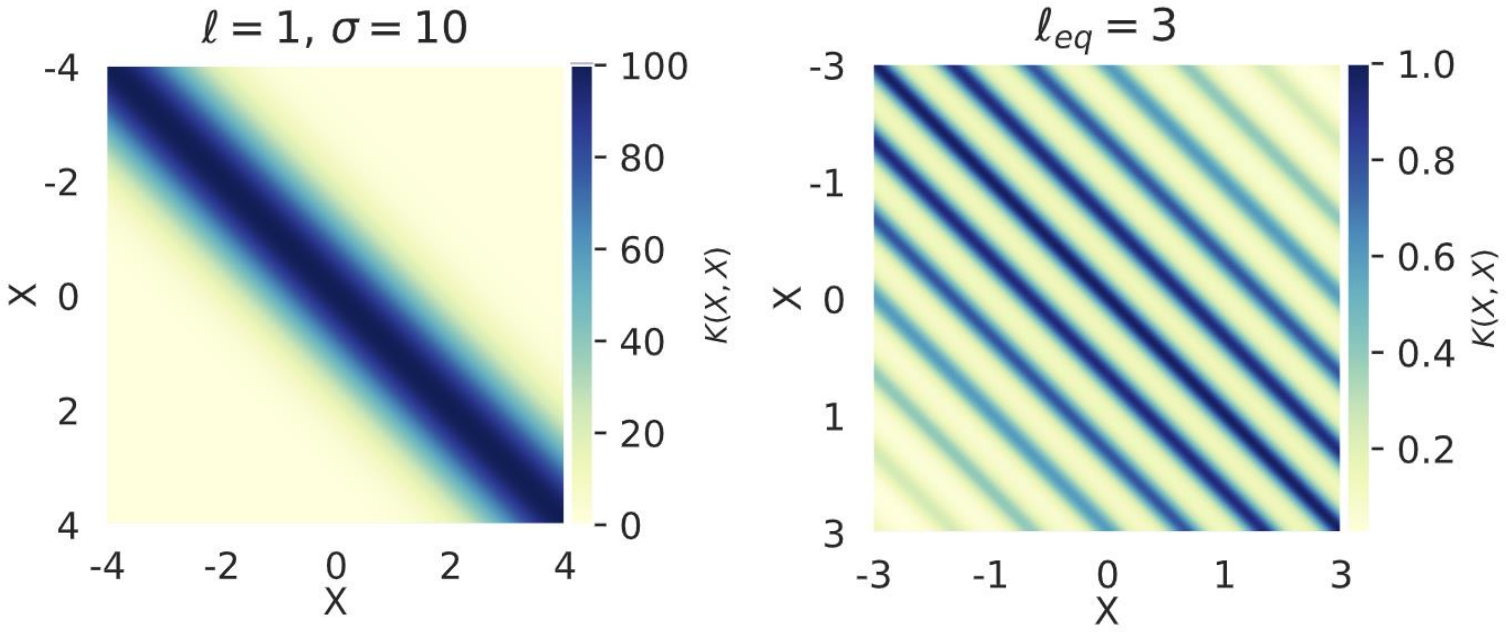
Kernel (covariance) matrices are usually generated from kernel (covariance) functions:

Square Exponential / RBF: $$k(x_1, x_2\,;\theta) = \theta_0 \exp\left(-\frac{(x_1 - x_2)^2}{2\theta_1^2}\right)$$

These are evaluated at the $\boldsymbol{X}, \boldsymbol{X}$ input vectors to create the matrix when algorithm is running

Valid covariance matrices must be **positive semi-definite**:

- Symmetric or anti-symmetric
- $\det(K) \geq 0$



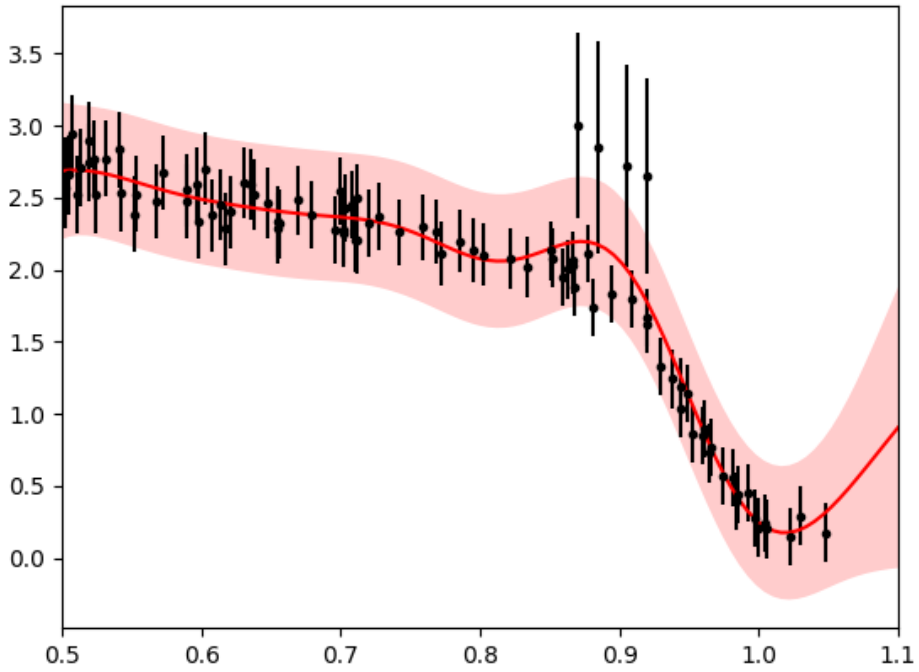
Kernel selection

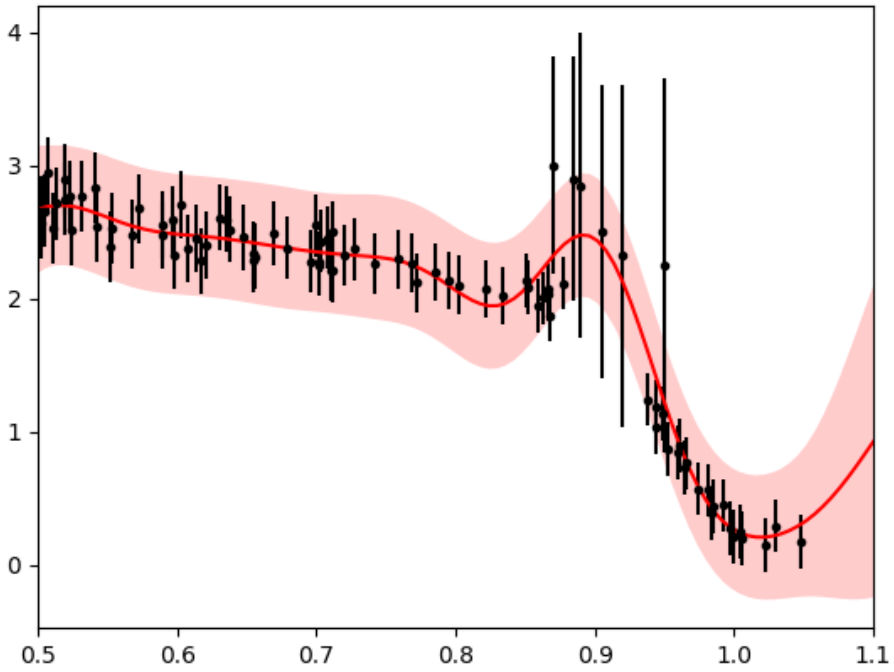
Known covariance functions:

- **Exponentiated quadratic / RBF**: standard kernel, single length scale, infinitely differentiable (very smooth)
- **Rational quadratic**: length scale blending, infinitely differentiable – recommended
- **Linear**: effectively $mx + b$, reduces GP to linear fitting, product of linear kernels give polynomials
- **Cylindrical**: cylindrical basis, good for data in polar coordinates
- **Periodic**: sin / cos basis, infinitely differentiable, greatly improves extrapolations of periodic data
- **Matérn**: user-defined differentiability (finer smoothness control)
- **Gibbs**: Function-based length scale, infinitely differentiable
- **Sigmoid**: emulates NN activation, good for resolving sharp boundaries and classification
- **Spectral Delta**: Similar to periodic but allows greater frequency control
- **Spectral Mixture**: Similar to RBF but allows greater control for smoothness

Outlier handling

- Since this techniques is based on Gaussian distributions, outliers heavily influence result
- Filtering as a pre-processing step is almost always necessary
- Some applications have used a **two-step** process: fit GP, exclude points outside bounds, fit GP again





Outlier handling

- Student-*t* Process has been developed which could be more robust to outliers
- Implementation seems not terribly difficult but there is no existing tool yet

Student-t Processes as Alternatives to Gaussian Processes

Amar Shah
University of Cambridge

Andrew Gordon Wilson
University of Cambridge

Zoubin Ghahramani
University of Cambridge

Abstract

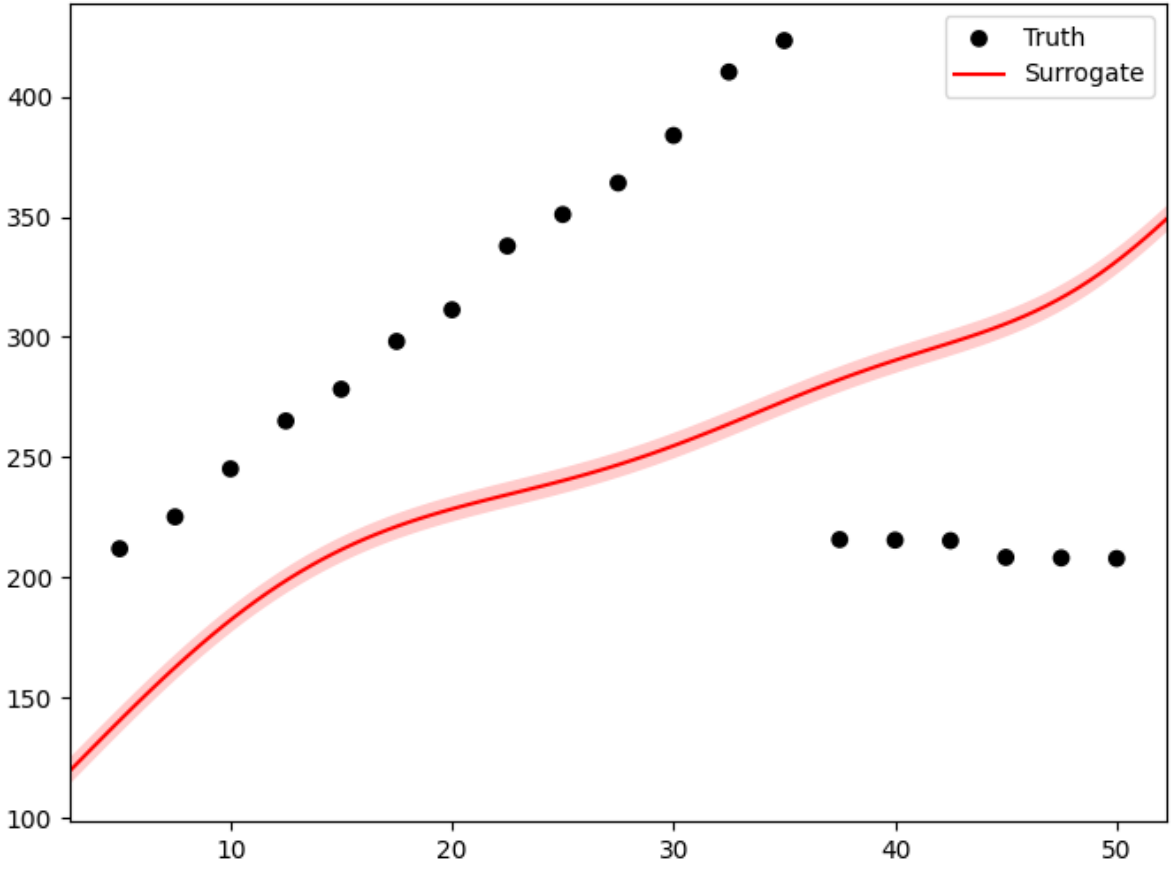
We investigate the Student-t process as an alternative to the Gaussian process as a nonparametric prior over functions. We derive closed form expressions for the marginal likelihood and predictive distribution of a Student-t process, by integrating away an inverse Wishart process prior over the covariance kernel of a Gaussian process model. We show surprising equivalences between different hierarchical Gaussian process models leading to Student-t processes, and derive a new sampling scheme for the inverse Wishart process, which helps elucidate these equivalences. Overall, we show that a Student-t process can retain the attractive properties of Gaussian processes as kernel machines have steadily grown in popularity over the last decade.

simple exact learning and inference procedures, and impressive empirical performances [Rasmussen, 1996],

At the heart of every Gaussian process (GP) is a parametrized covariance kernel, which determines the properties of likely functions under a GP. Typically simple parametric kernels, such as the Gaussian (squared exponential) kernel are used, and its parameters are determined through marginal likelihood maximization, having analytically integrated away the Gaussian process. However, a fully Bayesian nonparametric treatment of regression would place a nonparametric prior over the Gaussian process covariance kernel, to represent uncertainty over the kernel function, and to reflect the natural intuition that the kernel does not have a simple parametric form.

Backup

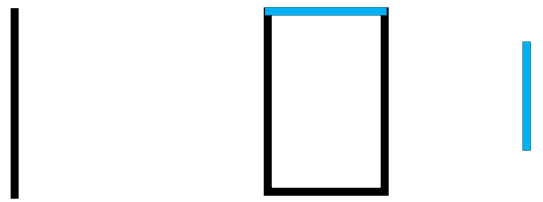
Issue with surrogate example



- A 1-D scan comparison reveals discontinuity in output space
  - Smooth kernel used in example cannot capture this feature while keeping smooth sections
- gpytorch implements one “kernel function” to be applied across all input dimensions
  - Smoothness learned from one dimension may not translate to others

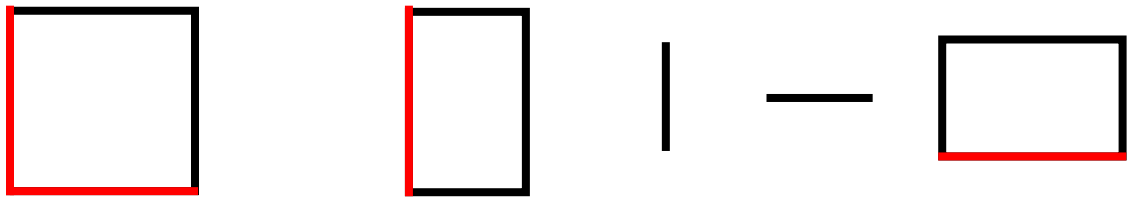
Abstracting out the basis functions

$$\boldsymbol{Y} = \boldsymbol{\Phi}(\boldsymbol{X}) \quad \boldsymbol{\beta}$$



After shift to probabilities

$$\boldsymbol{K} = \boldsymbol{\Phi}(\boldsymbol{X}) \quad \boldsymbol{\beta} \quad \boldsymbol{\beta}^{\boldsymbol{T}} \quad \boldsymbol{\Phi}^{\boldsymbol{T}}(\boldsymbol{X})$$



- Since GPs require computing $\boldsymbol{K}^{-1}$, the size of the matrix matters
- Since $\boldsymbol{\Phi}$ never appears outside of $\boldsymbol{K}$, the number of basis functions dimension can be ignored
  - Infinite basis function support (!)
- The size of $\boldsymbol{K}$ is determined solely by the number of training points, n
  - Covariance matrix inversion speed is $\sim\mathcal{O}(n^3)$

Welcome to the Machine Learning Working Group (MLWG)!

Objective:

Provide hands-on tutorials to get started with machine learning, e.g.:

- Manipulating / examining data
- Using widely available platforms
- Understanding popular ML modes
- Possibly more… (suggestions?)

PSFC members or external guests:

- If you wish to share something you are using, **we encourage you to lead a session!**

Slack: #machine-learning-working-group

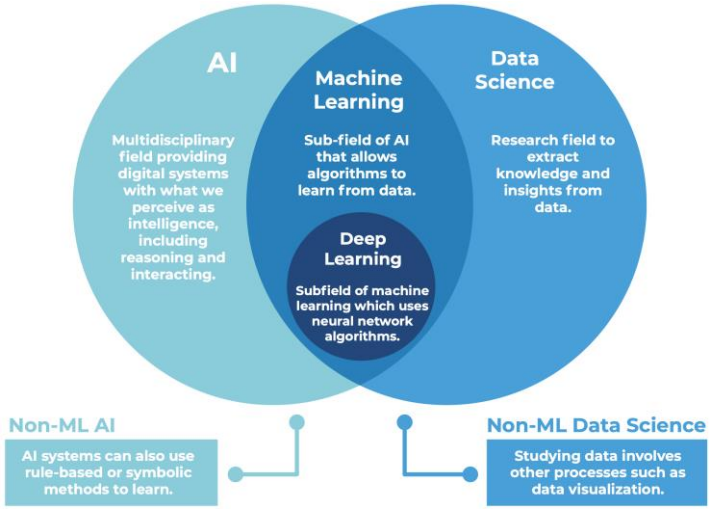
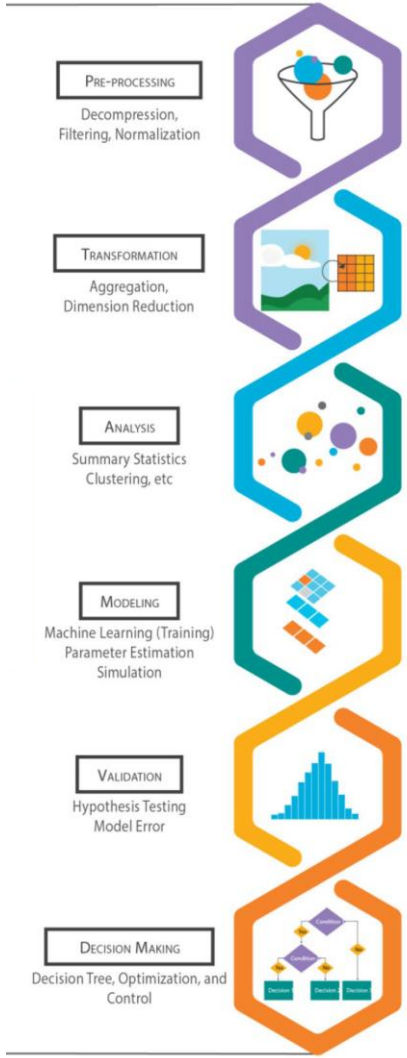


Image from https://www.ai-lawenforcement.org/guidance/techrefbook



[P. Kumar *et al.*, Sci Rep 10, 11492 (2020)]

Register Your Interest For The Sophelio Fusion Equilibrium Challenge 2025

Reactor-ready ML — No Magnetics Allowed

In future devices like SPARC and ARC, magnetic diagnostics will be sparse or absent. Can your model still infer flux surfaces, LCFS, and plasma shape — using only PF coil currents and Thomson scattering? Be the first to join the **APS 2025 Fusion Equilibrium Challenge**. Train on DIII-D, test on MAST. Score high in intra-machine fidelity and cross-device generalization.

The Task

Infer magnetic flux surfaces and core scalars — using only PF coil currents and Thomson profiles

The Data

Train on DIII-D, test on MAST. Real tokamak shots, real reactor challenges.

The Score

Accuracy + generalization: flux fidelity (R^2), LCFS match, scalar precision, cross-device transfer.

Join Early

Register now for early data access, submission details, and leaderboard launch.

https://datacomp.sophelio.io/